

ELF

Executable and Linkable Format

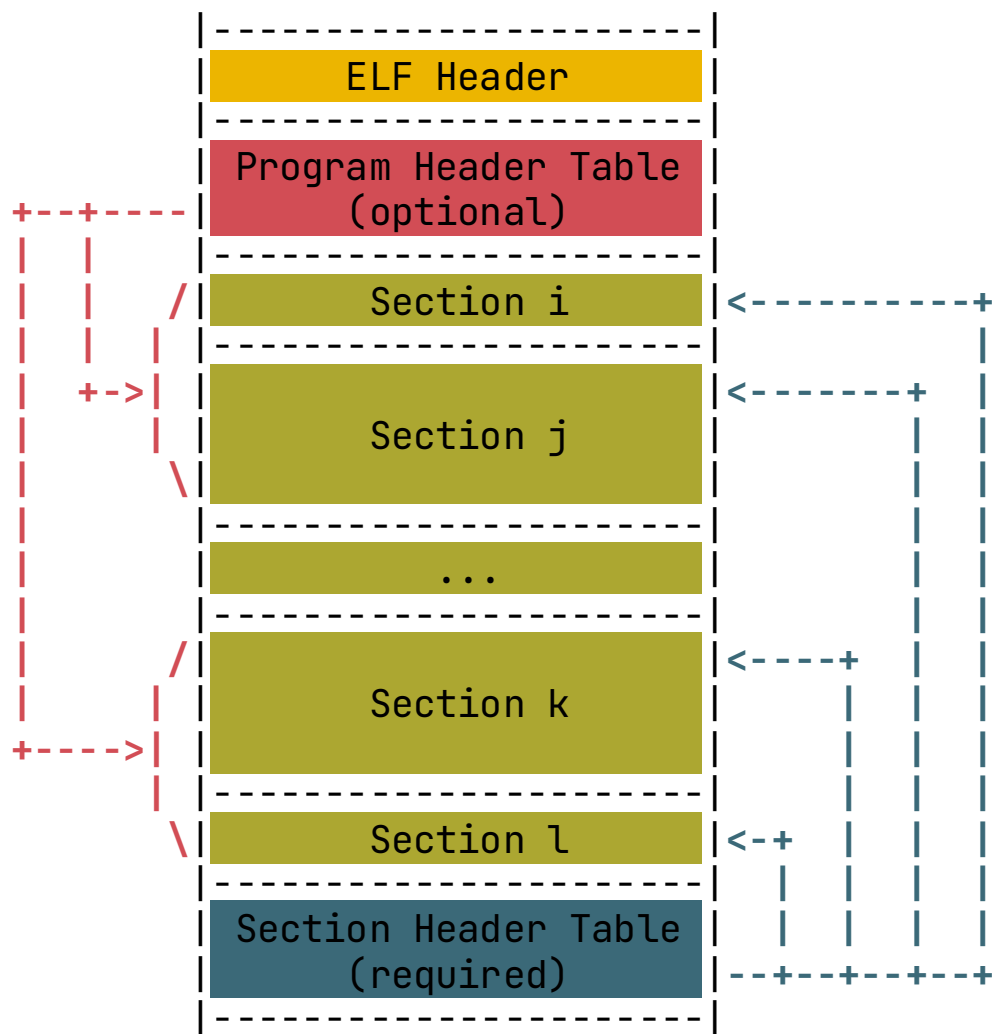
ELF

- *ELF* је стандардни формат објектних програма *UNIX-like* система
- *ELF* датотека представља бинарну репрезентацију програма намењених за извршавање директно на неком циљном процесору
- Разликујемо три типа *ELF* датотека са објектним програмом:
 - *relocatable ELF* датотека садржи машински код и податке у облику предвиђеном за повезивање са другим објектним програмима
 - *executable ELF* датотека садржи програм у облику погодном за извршавање
 - *shared object ELF* датотека садржи машински код и податке у облику предвиђеном за статичко и динамичко повезивање

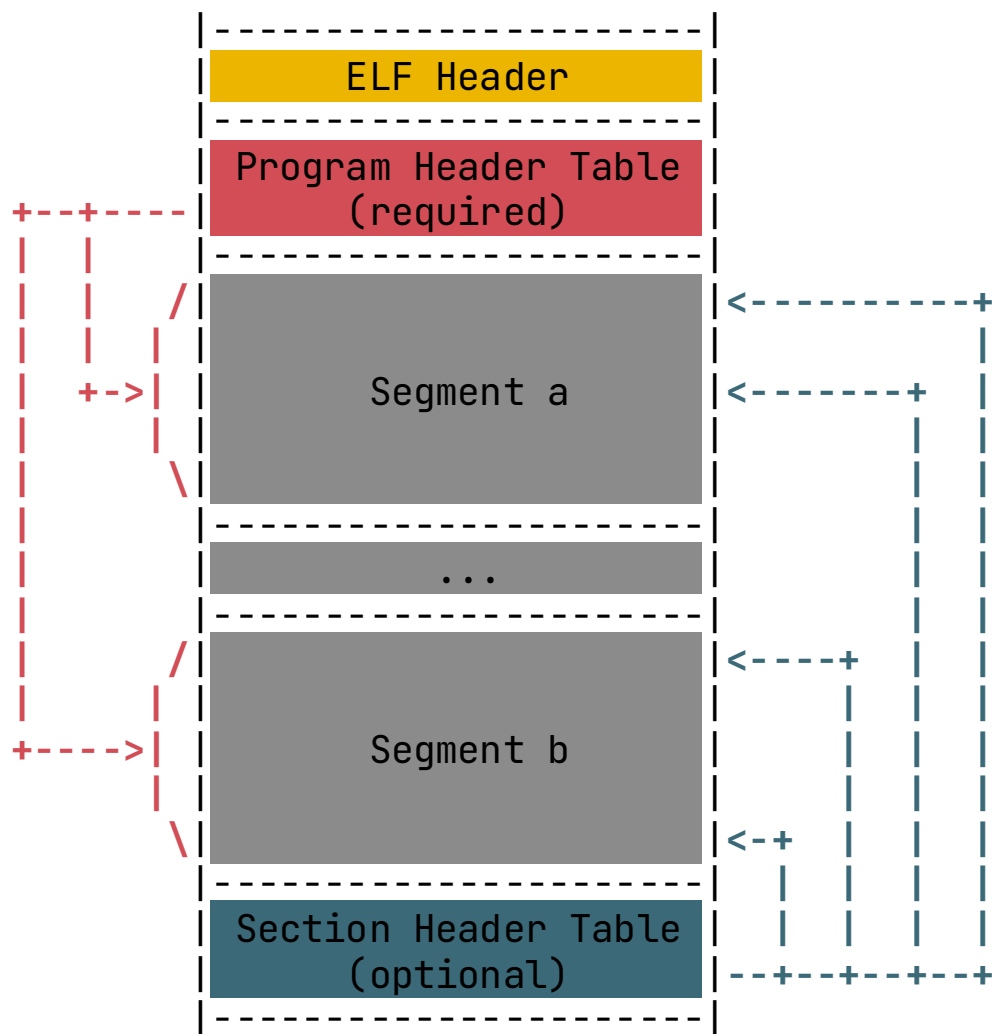
ELF формат

- На самом почетку сваке *ELF* датотеке налази се **једно обавезно заглавље односно *ELF header*** након чега се могу наћи следећи подаци у произвољном редоследу:
 - *section header table* са описом нула или више **секција**
 - *program header table* са описом нула или више **делова програма** (меморијски сегменти или информације за припрему извршавања)
 - садржај описан појединачним улазима унутар *section header table* и *program header table*
- *ELF* датотека услед тога што се користи у различитим процесима истовремено пружа могућност погледа на свој садржај из угла:
 - повезивања (*linking*)
 - извршавања (*execution*)

ELF формат (*linking view*)



ELF формат (*execution view*)



ELF формат

- *ELF* формат пружа подршку за различите процесоре са 32-битном (Elf32) и 64-битном (Elf64) архитектуром (*ELF* формат је проширив и за мање и за веће архитектуре)
- *ELF* формат експлицитно дефинише начин представљања података како би садржај *ELF* датотека било могуће исправно протумачити независно од машине на којој је садржај направљен (садржај је свакако намењен за неки циљни процесор)
- Сви типови података дефинисани *ELF* форматом поштују *System V ABI* правила по питању величине и поравнања
- Дефиницију *ELF* формата могуће је наћи на следећим местима:
 - *System V ABI (generic)* документација
 - системски приручник односно `man 5 elf`
 - директним увидом у изворни код `elf.h` заглавља

ELF типови података

- Посматраћемо *ELF* формат за 64-битну (Elf64) архитектуру
- Дефиниција елементарних типова података *ELF64* формата налази се у наставку:

Type	Size	Alignment	Purpose
-----	----	-----	-----
Elf64_Addr	8	8	Unsigned program address
Elf64_Off	8	8	Unsigned file offset
Elf64_Half	2	2	Unsigned medium integer
Elf64_Word	4	4	Unsigned integer
Elf64_Sword	4	4	Signed integer
Elf64_Xword	8	8	Unsigned long integer
Elf64_Sxword	8	8	Signed long integer
unsigned char	1	1	unsigned small integer

ELF типови података

- Дефиниција осталих типова података *ELF64* формата биће приказана на примеру *relocatable ELF* датотеке *hello.o* добијене асемблирањем изворног кода датог на следећем слајду
- *Relocatable ELF* датотеку *hello.o* могуће је добити на следећи начин:

```
as -o hello.o hello.s
```

- Бинарни садржај може се видети на следећи начин:

```
hd -v hello.o
```

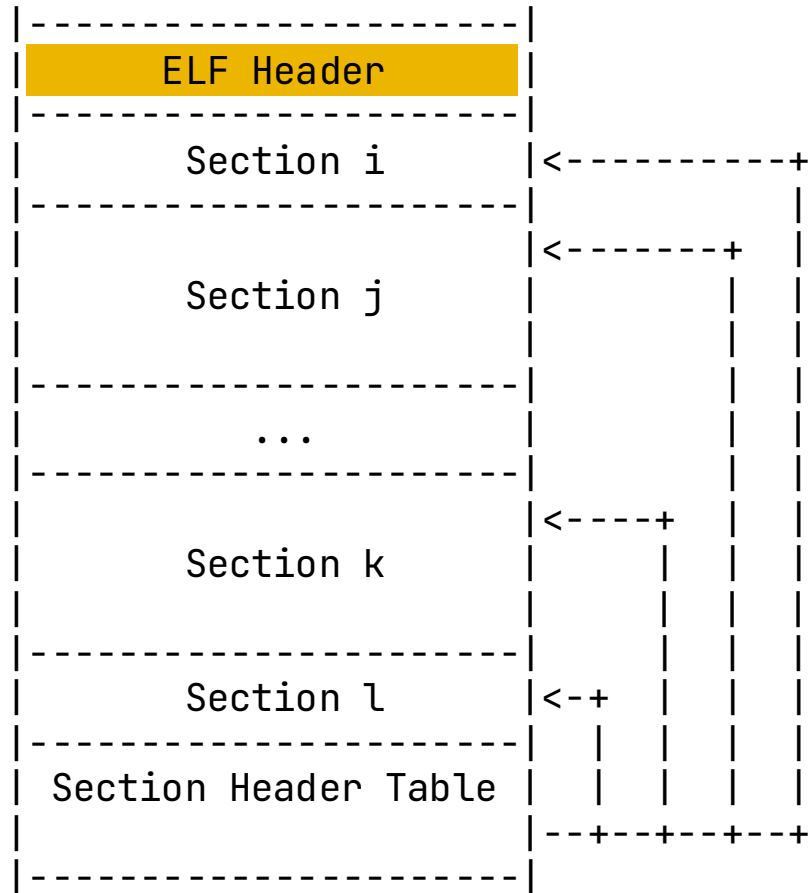
- Логички садржај може се видети на следећи начин:

```
objdump <option(s)> hello.o  
readelf <option(s)> hello.o
```

ELF типови података (пример *hello.s*)

```
.intel_syntax noprefix
.global custom_entry
.equ message_len, message_end - message_start
.data
message_start:
.asciz "Hello World!\n"
message_end:
.text
custom_entry:
    mov rax, 1           # write
    mov rdi, 1           # fd
    mov rsi, offset message_start # buf
    mov rdx, offset message_len  # count
    syscall
    mov rax, 60          # exit
    mov rdi, 13          # status
    syscall
.end
```

ELF Header



Elf64_Ehdr

```
#define EI_NIDENT 16
typedef struct {
    unsigned char    e_ident[EI_NIDENT];
    Elf64_Half       e_type;
    Elf64_Half       e_machine;
    Elf64_Word       e_version;
    Elf64_Addr       e_entry;
    Elf64_Off        e_phoff;
    Elf64_Off        e_shoff;
    Elf64_Word       e_flags;
    Elf64_Half       e_ehsize;
    Elf64_Half       e_phentsize;
    Elf64_Half       e_phnum;
    Elf64_Half       e_shentsize;
    Elf64_Half       e_shnum;
    Elf64_Half       e_shstrndx;
} Elf64_Ehdr;
```

Elf64_Ehdr

- Поља структуре `Elf64_Ehdr`, која представљају *ELF header*, описаћемо појединачно на примеру *ELF* датотеке *hello.o*
- Садржај структуре `Elf64_Ehdr` у *relocatable ELF* датотеци *hello.o* приказан је на дну слајда

00000000:	7f 45 4c 46 02 01 01 00	00 00 00 00 00 00 00 00
00000010:	01 00 3e 00 01 00 00 00	00 00 00 00 00 00 00 00
00000020:	00 00 00 00 00 00 00 00	c8 01 00 00 00 00 00 00
00000030:	00 00 00 00 40 00 00 00	00 00 40 00 08 00 07 00

Elf64_Ehdr.e_ident[]

- Низ од 16 елемената на самом почетку *ELF* датотеке:
 - прва 4 елемента представљају ознаку *ELF* датотеке (*magic number*)
 - преостали елементи пружају информацију о томе како треба декодовати и интерпретирати садржај *ELF* датотеке (омогућено је тумачење *ELF* датотеке на било којој архитектури)
- ELF формат дефинише константе за индексирање овог низа:

```
#define EI_MAG0      0  /* File identification byte 0 index */
#define EI_MAG1      1  /* File identification byte 1 index */
#define EI_MAG2      2  /* File identification byte 2 index */
#define EI_MAG3      3  /* File identification byte 3 index */
#define EI_CLASS      4  /* File class byte index */
#define EI_DATA       5  /* Data encoding byte index */
#define EI_VERSION    6  /* File version byte index */
#define EI_OSABI      7  /* OS ABI identification */
#define EI_ABIVERSION  8  /* ABI version */
#define EI_PAD        9  /* Byte index of padding bytes */
```

Elf64_Ehdr.e_ident[EI_MAGn]

- Прва 4 елемента низа e_ident представљају „магични број“ којим се означава *ELF* датотека:
`0x7f, 'E', 'L', 'F'`

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_ident[EI_CLASS]

- Поље `e_ident[EI_CLASS]` говори да ли је реч о 32-битној или 64-битној архитектури
- *ELF* формат дефинише следеће могуће вредности:

```
#define ELFCLASSNONE 0 /* Invalid class */  
#define ELFCLASS32 1 /* 32-bit objects */  
#define ELFCLASS64 2 /* 64-bit objects */
```

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00  
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00  
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_ident[EI_DATA]

- Поље `e_ident[EI_DATA]` специфицира начин декодовања *ELF* датотеке односно како су смештене вишебајтне вредности
- *ELF* формат дефинише следеће могуће вредности:

```
#define ELFDATANONE 0 /* Invalid data encoding */  
#define ELFDATA2LSB 1 /* 2's complement, little endian */  
#define ELFDATA2MSB 2 /* 2's complement, big endian */
```

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00  
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00  
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_ident[EI_VERSION]

- Поље `e_ident[EI_VERSION]` дефинише верзију *ELF* формата
- *ELF* формат дефинише следеће могуће вредности:

```
#define EV_NONE      0  /* Invalid ELF version */  
#define EV_CURRENT  1  /* Current version */
```

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00  
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00  
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_ident[EI_OSABI]

- Поље `e_ident[EI_OSABI]` специфицира коју *ELF* екстензију користи посматрана *ELF* датотека у погледу оперативног система и *ABI*
- *ELF* формат дефинише следеће могуће вредности:

```
#define ELFOSABI_NONE      0 /* UNIX System V ABI */
#define ELFOSABI_SYSV      0 /* Alias. */
#define ELFOSABI_HPUX      1 /* HP-UX */
#define ELFOSABI_NETBSD    2 /* NetBSD. */
#define ELFOSABI_GNU       3 /* Object uses GNU ELF extensions. */
...
```

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_ident[EI_ABIVERSION]

- Поље `e_ident[EI_ABIVERSION]` дефинише верзију *ABI* за коју је *ELF* датотека направљена
- Тумачење овог елемента зависи од коришћеног *ABI*

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_ident[EI_PAD]

- Поље `e_ident[EI_PAD]` представља први елемент у низу неискоришћених елемената низа `e_ident`
- Сваки елемент низа `e_ident` почев од индекса `EI_PAD` треба да има вредност 0 (нула)

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_type

- Поље e_type дефинише тип *ELF* датотеке
- *ELF* формат дефинише следеће могуће вредности:

```
#define ET_NONE          0  /* No file type */
#define ET_REL           1  /* Relocatable file */
#define ET_EXEC          2  /* Executable file */
#define ET_DYN           3  /* Shared object file */
#define ET_CORE          4  /* Core file */
#define ET_LOOS          0xfe00 /* OS-specific range start */
#define ET_HIOS          0xfeff /* OS-specific range end */
#define ET_LOPROC        0xff00 /* Processor-specific range start */
#define ET_HIPROC        0xffff /* Processor-specific range end */
```

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_machine

- Поље `e_machine` специфицира коју архитектуру захтева *ELF* датотека
- *ELF* формат дефинише следеће могуће вредности:

```
#define EM_NONE      0  /* No machine */
#define EM_386       3  /* Intel 80386 */
#define EM_860       7  /* Intel 80860 */
#define EM_ARM       40 /* ARM */
#define EM_MIPS_X    51 /* Stanford MIPS-X */
#define EM_X86_64    62 /* AMD x86-64 architecture */
#define EM_RISCV     243 /* RISC-V */
...
```

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_version

- Поље `e_version` дефинише верзију *ELF* формата
- *ELF* формат дефинише следеће могуће вредности:

```
#define EV_NONE      0  /* Invalid ELF version */  
#define EV_CURRENT  1  /* Current version */
```

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00  
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00  
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_entry

- Поље e_entry дефинише улазну тачку програма односно виртуелну адресу почев од које ће систем започети извршавање

00000000:	7f	45	4c	46	02	01	01	00	00	00	00	00	00	00	00	00
00000010:	01	00	3e	00	01	00	00	00	00	00	00	00	00	00	00	00
00000020:	00	00	00	00	00	00	00	00	c8	01	00	00	00	00	00	00
00000030:	00	00	00	00	40	00	00	00	00	00	40	00	08	00	07	00

Elf64_Ehdr.e_phoff

- Поље `e_phoff` дефинише *program header table offset* односно померај од почетка *ELF* датотеке изражено у броју бајтова до локације где се налази *program header table* (*program header table* може се наћи било где у *ELF* датотеци)
- Уколико *program header table* не постоји у *ELF* датотеци ово поље треба да садржи вредност 0 (нула)

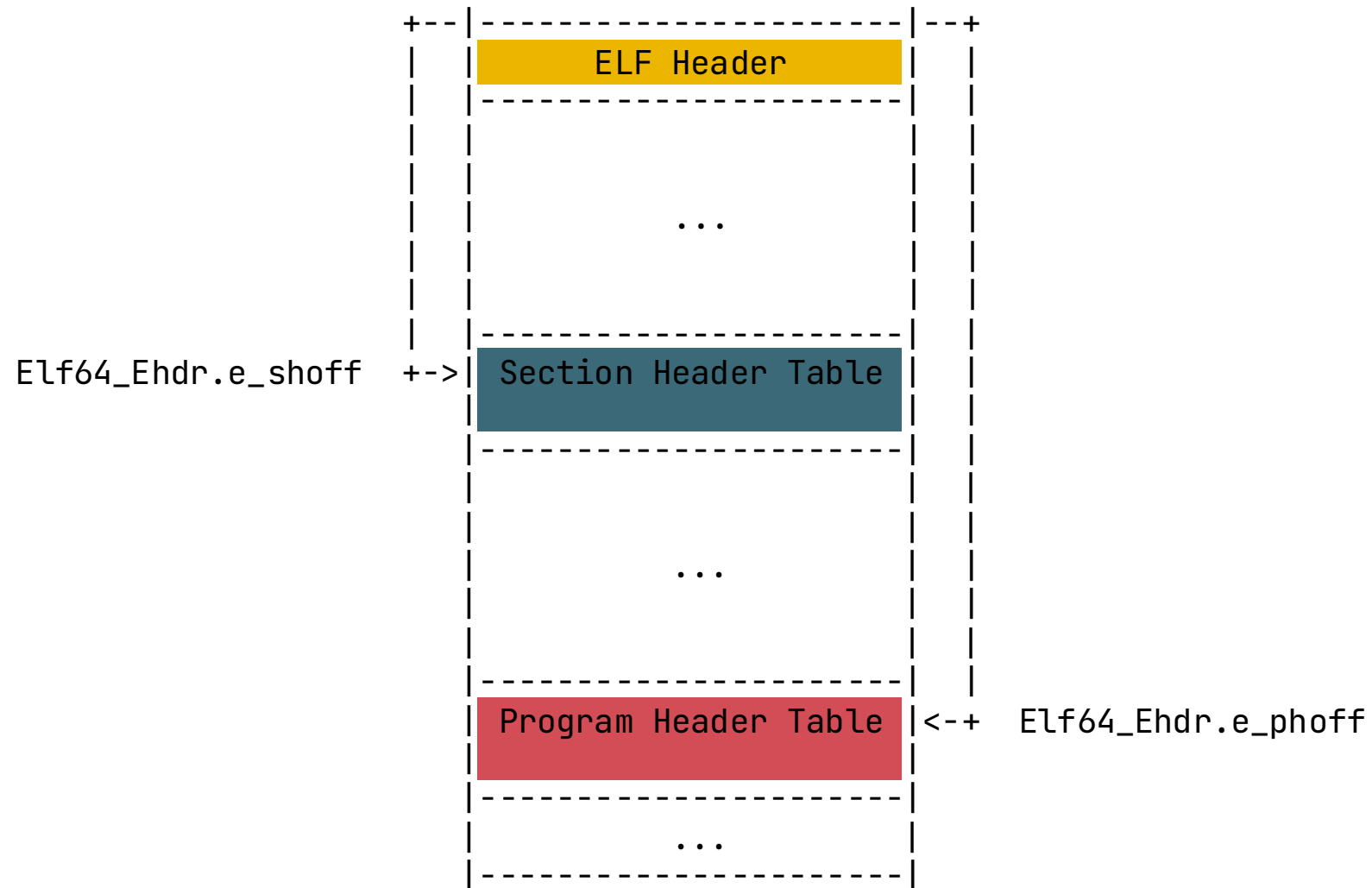
```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_shoff

- Поље `e_shoff` дефинише *section header table offset* односно померај од почетка *ELF* датотеке изражено у броју бајтова до локације где се налази *section header table* (*section header table* може се наћи било где у *ELF* датотеци)
- Уколико *section header table* не постоји у *ELF* датотеци ово поље треба да садржи вредност 0 (нула)

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_xhoffs



Elf64_Ehdr.e_flags

- Поље `e_flags` наводи флегове специфичне за посматрани процесор
- *ELF* формат дефинише могуће вредности у следећем формату:
`EF_<machine>_<flag>`

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_ehsize

- Поље `e_ehsize` дефинише величину *ELF header* изражено у броју бајтова

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_phentsize

- Поље `e_phentsize` дефинише величину једног улаза у *program header table* изражено у броју бајтова (сви улази су исте величине)

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_phnum

- Поље e_phnum представља број улаза у *program header table*

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_shentsize

- Поље `e_shentsize` дефинише величину једног улаза у *section header table* изражено у броју бајтова (сви улази су исте величине)

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_shnum

- Поље e_shnum представља број улаза у *section header table*

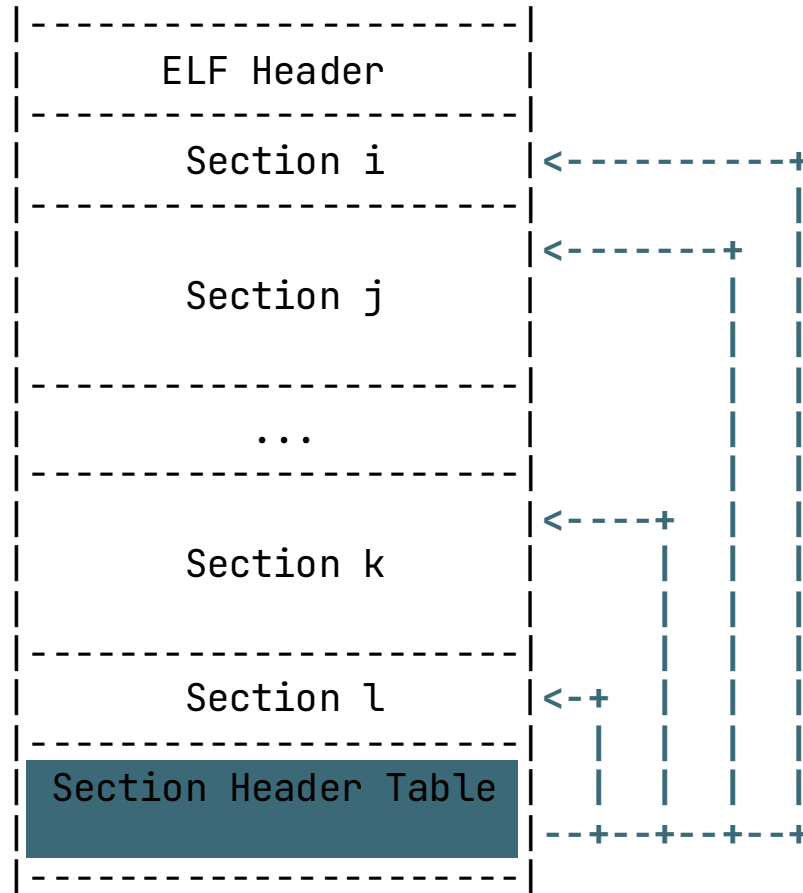
```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Elf64_Ehdr.e_shstrndx

- Поље `e_shstrndx` специфицира индекс улаза у *section header table* који описује секцију са стринговима за имена секција (*section name string table*)

```
00000000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
00000010: 01 00 3e 00 01 00 00 00 00 00 00 00 00 00 00 00
00000020: 00 00 00 00 00 00 00 00 c8 01 00 00 00 00 00 00
00000030: 00 00 00 00 40 00 00 00 00 00 40 00 08 00 07 00
```

Section Header Table



Section Header Table

- *Section header table* представља низ елемената типа `Elf64_Shdr`
- Елементи *section header table* уобичајено се називају „улази“; сваки улаз представља заглавље односно опис једне секције (улаз у *section header table* = заглавље секције = опис секције)
- На основу садржаја *ELF header* могуће је добити следеће податке:
 - `Elf64_Ehdr.e_shoff` је померај од почетка *ELF* датотеке до локације где почиње *section header table*
 - `Elf64_Ehdr.e_shnum` је број елемената односно улаза унутар *section header table*
 - `Elf64_Ehdr.e_shentsize` је величина једног елемента односно улаза унутар *section header table*

Section Header Table

- Поједини улази у *section header table* сматрају се резервисаним; индекси резервисаних односно специфичних заглавља секција дати су у наставку:

```
#define SHN_UNDEF          0      /* Undefined section */
#define SHN_LORESERVE    0xff00  /* Start of reserved indices */
#define SHN_LOPROC       0xff00  /* Start of processor-specific */
#define SHN_HIPROC       0xff1f  /* End of processor-specific */
#define SHN_LOOS         0xff20  /* Start of OS-specific */
#define SHN_HIOS         0xff3f  /* End of OS-specific */
#define SHN_ABS          0xffff1  /* Associated symbol is absolute */
#define SHN_COMMON       0xffff2  /* Associated symbol is common */
#define SHN_XINDEX       0xfffff  /* Index is in extra table. */
#define SHN_HIRESERVE    0xfffff  /* End of reserved indices */
```

Section Header Table

- **Секције** садрже све податке унутар *ELF* датотеке поред података из *ELF header*, *section header table* и *program header table*
- За **секције** важи следеће:
 - свака секција унутар *ELF* датотеке има свој улаз у *section header table* (могу постојати улази у *section header table* за које не постоји секција)
 - свака секција обухвата континуални низ бајтова у *ELF* датотеци
 - секције се не преклапају у *ELF* датотеци
 - поједини делови *ELF* датотеке теоријски могу бити неактивни (тако што не припадају ниједној секцији)

Section Header Table

- На основу вредности поља `Elf64_Ehdr.e_shoff` добија се информација где се налази *section header table* гледано у односу на почетак *ELF* датотеке
- Садржај *section header table* у *relocatable ELF* датотеци *hello.o*, чије поље `Elf64_Ehdr.e_shoff` има вредност `0x1C8`, приказан је на следећим слајдовима

Section Header Table (hello.o)

000001c8:	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	\	Index 0 (res)
000001d8:	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
000001e8:	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
000001f8:	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	/	
00000208:	20 00 00 00 01 00 00 00	06 00 00 00 00 00 00 00	\	Index 1
00000218:	00 00 00 00 00 00 00 00	40 00 00 00 00 00 00 00		
00000228:	2e 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
00000238:	01 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	/	
00000248:	1b 00 00 00 04 00 00 00	40 00 00 00 00 00 00 00	\	Index 2
00000258:	00 00 00 00 00 00 00 00	78 01 00 00 00 00 00 00		
00000268:	18 00 00 00 00 00 00 00	05 00 00 00 01 00 00 00		
00000278:	08 00 00 00 00 00 00 00	18 00 00 00 00 00 00 00	/	
00000288:	26 00 00 00 01 00 00 00	03 00 00 00 00 00 00 00	\	Index 3
00000298:	00 00 00 00 00 00 00 00	6e 00 00 00 00 00 00 00		
000002a8:	0e 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
000002b8:	01 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	/	

Section Header Table (hello.o)

000002c8:	2c 00 00 00 08 00 00 00	03 00 00 00 00 00 00 00	\	
000002d8:	00 00 00 00 00 00 00 00	7c 00 00 00 00 00 00 00		Index 4
000002e8:	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
000002f8:	01 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	/	
00000308:	01 00 00 00 02 00 00 00	00 00 00 00 00 00 00 00	\	
00000318:	00 00 00 00 00 00 00 00	80 00 00 00 00 00 00 00		Index 5
00000328:	c0 00 00 00 00 00 00 00	06 00 00 00 07 00 00 00		
00000338:	08 00 00 00 00 00 00 00	18 00 00 00 00 00 00 00	/	
00000348:	09 00 00 00 03 00 00 00	00 00 00 00 00 00 00 00	\	
00000358:	00 00 00 00 00 00 00 00	40 01 00 00 00 00 00 00		Index 6
00000368:	34 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
00000378:	01 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	/	
00000388:	11 00 00 00 03 00 00 00	00 00 00 00 00 00 00 00	\	
00000398:	00 00 00 00 00 00 00 00	90 01 00 00 00 00 00 00		Index 7
000003a8:	31 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00		
000003b8:	01 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	/	

Elf64_Shdr

```
typedef struct {  
    Elf64_Word    sh_name;  
    Elf64_Word    sh_type;  
    Elf64_Xword   sh_flags;  
    Elf64_Addr    sh_addr;  
    Elf64_Off     sh_offset;  
    Elf64_Xword   sh_size;  
    Elf64_Word    sh_link;  
    Elf64_Word    sh_info;  
    Elf64_Xword   sh_addralign;  
    Elf64_Xword   sh_entsize;  
} Elf64_Shdr;
```

Elf64_Shdr

- Поља структуре `Elf64_Shdr`, која чине заглавље једне секције, описаћемо појединачно на примеру једног конкретног заглавља секције из *ELF* датотеке *hello.o*
- Садржај структуре `Elf64_Shdr` у *relocatable ELF* датотеци *hello.o* која одговара улазу у *section header table* са индексом 2 (два) приказан је на дну слајда

```
00000248: 1b 00 00 00 04 00 00 00
00000258: 00 00 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00
00000278: 08 00 00 00 00 00 00 00
```

```
40 00 00 00 00 00 00 00
78 01 00 00 00 00 00 00
05 00 00 00 01 00 00 00
18 00 00 00 00 00 00 00
```

Elf64_Shdr.sh_name

- Поље `sh_name` представља померај унутар *section name string table* односно померај унутар секције са стринговима за имена секција
- На основу садржаја *section name string table* датог у наставку, до којег се долази индиректно преко поља `Elf64_Ehdr.e_shstrndx`, могуће је увидети да је име посматране секције **`.rela.text`**

```
00000190: 00 2e 73 79 6d 74 61 62 00 2e 73 74 72 74 61 62
000001a0: 00 2e 73 68 73 74 72 74 61 62 00 2e 72 65 6c 61
000001b0: 2e 74 65 78 74 00 2e 64 61 74 61 00 2e 62 73 73
000001c0: 00
```

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Elf64_Shdr.sh_type

- Поље sh_type је тип секције који дефинише њен садржај и значење
- *ELF* формат дефинише следеће могуће вредности:

```
#define SHT_NULL      0  /* Section header table entry unused */
#define SHT_PROGBITS  1  /* Program data */
#define SHT_SYMTAB    2  /* Symbol table */
#define SHT_STRTAB    3  /* String table */
#define SHT_RELA      4  /* Relocation entries with addends */
#define SHT_DYNAMIC   6  /* Dynamic linking information */
#define SHT_NOTE      7  /* Notes */
#define SHT_NOBITS    8  /* Program space with no data (bss) */
#define SHT_REL       9  /* Relocation entries, no addends */
...
```

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Elf64_Shdr.sh_flags

- Поље sh_flags представља маску једнобитних флегова секције
- *ELF* формат дефинише следеће могуће вредности:

```
#define SHF_WRITE      (1 << 0)  /* Writable */
#define SHF_ALLOC      (1 << 1)  /* Occupies memory during execution */
#define SHF_EXECINSTR  (1 << 2)  /* Executable */
#define SHF_MERGE      (1 << 4)  /* Might be merged */
#define SHF_STRINGS    (1 << 5)  /* Contains nul-terminated strings */
#define SHF_INFO_LINK  (1 << 6)  /* 'sh_info' contains SHT index */
#define SHF_GROUP      (1 << 9)  /* Section is member of a group. */
#define SHF_TLS        (1 << 10) /* Section hold thread-local data. */
#define SHF_MASKOS     0x0ff00000 /* OS-specific. */
#define SHF_MASKPROC   0xf0000000 /* Processor-specific */
```

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Elf64_Shdr.sh_addr

- Поље sh_addr представља виртуелну адресу првог бајта **секције** уколико дата секција треба да буде у меморији у току извршавања док у супротном ово поље има вредност 0 (нула)

00000248:	1b 00 00 00 04 00 00 00	40 00 00 00 00 00 00 00
00000258:	00 00 00 00 00 00 00 00	78 01 00 00 00 00 00 00
00000268:	18 00 00 00 00 00 00 00	05 00 00 00 01 00 00 00
00000278:	08 00 00 00 00 00 00 00	18 00 00 00 00 00 00 00

Elf64_Shdr.sh_offset

- Поље `sh_offset` представља померај од почетка *ELF* датотеке до првог бајта посматране **секције** изражено у броју бајтова

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Elf64_Shdr.sh_size

- Поље `sh_size` представља величину **секције** у бајтовима
- Величину различиту од нуле могу имати чак и секције које не заузимају простор у *ELF* датотеци (секције чији је тип `SHT_NOBITS`)

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Elf64_Shdr.sh_link

- Поље `sh_link` је индекс неког другог улаза у *section header table* са којим је посматрани улаз у *section header table* у некаквој вези (интерпретација ове везе зависи од типа посматране секције)

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Elf64_Shdr.sh_info

- Поље sh_info садржи произвољан додатни податак
(интерпретација овог податка зависи од типа посматране секције)

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Elf64_Shdr

Elf64_Shdr.sh_type	Elf64_Shdr.sh_link	Elf64_Shdr.sh_info
SHT_DYNAMIC	Индекс заглавља табеле стрингова коју користе улази у овој секцији	0
SHT_HASH	Индекс заглавља табеле симбола на коју се примењује <i>hash table</i>	0
SHT_REL SHT_RELA	Индекс заглавља придружене табеле симбола	Индекс заглавља секције на коју се примењују релокације
SHT_SYMTAB SHT_DYNSYM	Индекс заглавља придружене табеле стрингова	Вредност за један већа од индекса последњег локалног симбола
други типови	SHN_UNDEF	0

Elf64_Shdr.sh_addralign

- Поље sh_addralign дефинише поравнање **секције**
 - нула или један значи да није захтевано поравнање
 - поред нула и један дозвољене су само вредности које су степен двојке

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Elf64_Shdr.sh_entsize

- Поље sh_entsize представља величину једног улаза унутар секције; неке **секције** садрже табеле са улазима фиксних величина као што су, на пример, табела релокација и табела симбола
- Уколико садржај секције није табела са улазима фиксних величина вредност поља треба да буде 0 (нула)

```
00000248: 1b 00 00 00 04 00 00 00 40 00 00 00 00 00 00 00
00000258: 00 00 00 00 00 00 00 00 78 01 00 00 00 00 00 00
00000268: 18 00 00 00 00 00 00 00 05 00 00 00 01 00 00 00
00000278: 08 00 00 00 00 00 00 00 18 00 00 00 00 00 00 00
```

Специјалне секције (1/3)

Назив	Тип	Атрибути
.bss	SHT_NOBITS	SHF_ALLOC+SHF_WRITE
.comment	SHT_PROGBITS	
.data	SHT_PROGBITS	SHF_ALLOC+SHF_WRITE
.data1	SHT_PROGBITS	SHF_ALLOC+SHF_WRITE
.debug	SHT_PROGBITS	
.dynamic	SHT_DYNAMIC	
.dynstr	SHT_STRTAB	SHF_ALLOC
.dynsym	SHT_DYNSYM	SHF_ALLOC
.fini	SHT_PROGBITS	SHF_ALLOC+SHF_EXECINSTR
.fini_array	SHT_FINI_ARRAY	SHF_ALLOC+SHF_WRITE
.got	SHT_PROGBITS	

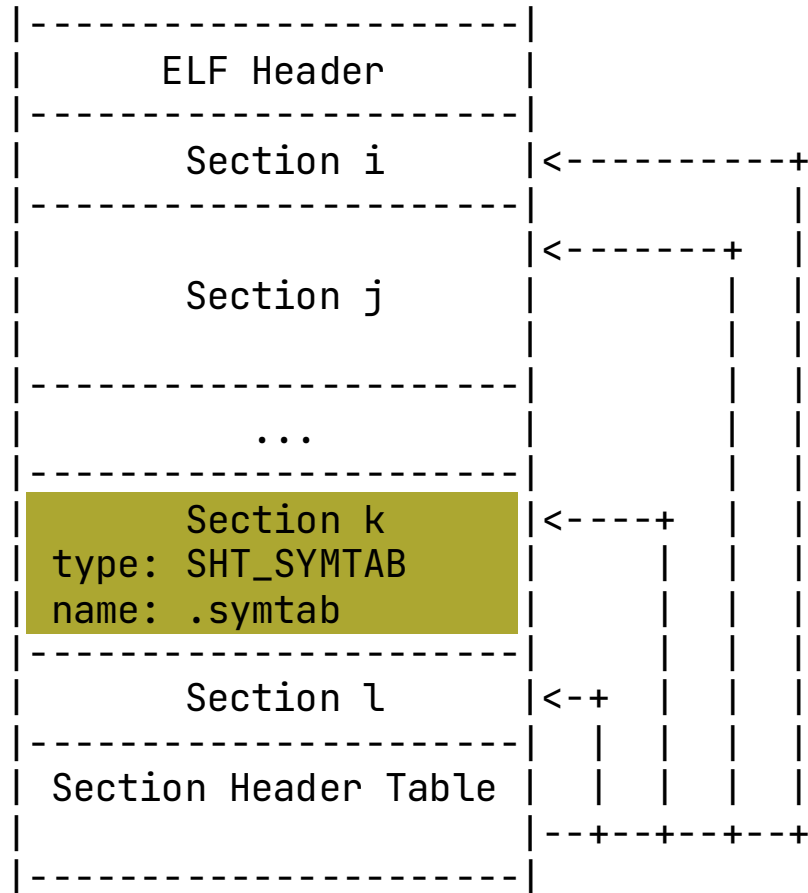
Специјалне секције (2/3)

Назив	Тип	Атрибути
.hash	SHT_HASH	SHF_ALLOC
.init	SHT_PROGBITS	SHF_ALLOC+SHF_EXECINSTR
.init_array	SHT_INIT_ARRAY	SHF_ALLOC+SHF_WRITE
.interp	SHT_PROGBITS	
.line	SHT_PROGBITS	
.note	SHT_NOTE	
.plt	SHT_PROGBITS	
.preinit_array	SHT_PREINIT_ARRAY	SHF_ALLOC+SHF_WRITE
.relname	SHT_REL	
.relaname	SHT_RELA	
.rodata	SHT_PROGBITS	SHF_ALLOC

Специјалне секције (3/3)

Назив	Тип	Атрибути
.rodata1	SHT_PROGBITS	SHF_ALLOC
.shstrtab	SHT_STRTAB	
.strtab	SHT_STRTAB	
.symtab	SHT_SYMTAB	
.symtab_shndx	SHT_SYMTAB_SHNDX	
.tbss	SHT_NOBITS	SHF_ALLOC+SHF_WRITE+SHF_TLS
.tdata	SHT_PROGBITS	SHF_ALLOC+SHF_WRITE+SHF_TLS
.tdata1	SHT_PROGBITS	SHF_ALLOC+SHF_WRITE+SHF_TLS
.text	SHT_PROGBITS	SHF_ALLOC+SHF_EXECINSTR

Symbol Table



Symbol Table

- *Symbol table* је **секција** типа SHN_SYMTAB чији је садржај табела са улазима фиксних величина који су типа Elf64_Sym (сваки појединачни улаз представља један симбол)
- На основу садржаја заглавља секције типа SHN_SYMTAB могуће је добити следеће податке:
 - Elf64_Shdr.sh_offset је померај од почетка *ELF* датотеке до локације где почиње *symbol table*
 - Elf64_Shdr.sh_size је величина секције *symbol table* изражено у броју бајтова
 - Elf64_Shdr.sh_link је индекс заглавља секције придружене табеле стрингова
 - Elf64_Shdr.sh_info је вредност за један већа од индекса последњег локалног симбола унутар *symbol table*
 - Elf64_Shdr.sh_entsize је величина једног елемента односно улаза унутар *symbol table*

Symbol Table

- На основу вредности поља `Elf64_Shdr.sh_offset` унутар заглавља секције типа `SHT_SYMTAB` (*ELF* датотека може имати само једну секцију типа `SHT_SYMTAB`) добија се информација где се налази *symbol table* гледано у односу на почетак *ELF* датотеке
- Садржај *symbol table* у *relocatable ELF* датотеци *hello.o*, где унутар заглавља секције типа `SHT_SYMTAB` поље `Elf64_Shdr.sh_offset` има вредност `0x80` а поље `Elf64_Shdr.sh_size` има вредност `0xc0`, приказан је на следећем слајду

Symbol Table (hello.o)

00000080:	00	00	00	00	00	00	00	00
00000090:	00	00	00	00	00	00	00	00
000000a0:	00	00	00	00	00	00	00	00
000000b0:	00	00	00	00	03	00	03	00
000000c0:	00	00	00	00	00	00	00	00
000000d0:	00	00	00	00	00	00	00	00
000000e0:	01	00	00	00	00	00	f1	ff
000000f0:	00	00	00	00	00	00	00	00
00000100:	0e	00	00	00	00	00	00	00
00000110:	19	00	00	00	00	00	03	00
00000120:	00	00	00	00	00	00	00	00
00000130:	00	00	00	00	00	00	00	00

00	00	00	00	00	00	00	00
00	00	00	00	03	00	01	00
00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00
00	00	00	00	03	00	04	00
00	00	00	00	00	00	00	00
0e	00	00	00	00	00	00	00
0d	00	00	00	00	00	03	00
00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00
27	00	00	00	10	00	01	00
00	00	00	00	00	00	00	00

Elf64_Sym

```
typedef struct {  
    Elf64_Word      st_name;  
    unsigned char    st_info;  
    unsigned char    st_other;  
    Elf64_Half       st_shndx;  
    Elf64_Addr       st_value;  
    Elf64_Xword       st_size;  
} Elf64_Sym;
```

Elf64_Sym

- Поља структуре `Elf64_Sym`, која дефинишу један симбол, описаћемо појединачно на примеру једног конкретног симбола из *ELF* датотеке *hello.o*
- Садржај структуре `Elf64_Sym` у *relocatable ELF* датотеци *hello.o* која одговара улазу у *symbol table* са индексом 5 (пет) приказан је на дну слајда

000000e0:	01 00 00 00 00 00 f1 ff	0e 00 00 00 00 00 00 00
000000f0:	00 00 00 00 00 00 00 00	0d 00 00 00 00 00 03 00
00000100:	0e 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00
00000110:	19 00 00 00 00 00 03 00	00 00 00 00 00 00 00 00

Elf64_Sym.st_name

- Поље `st_name` представља померај унутар *symbol string table* односно померај унутар придружене секције са стринговима (индекс заглавља ове секције специфициран је пољем `sh_link`)
- На основу садржаја *symbol string table* датог у наставку, до којег се долази индиректно преко поља `Elf64_Shdr.sh_link`, могуће је увидети да је име посматраног симбола **message_end**

```
00000140: 00 6d 65 73 73 61 67 65 5f 6c 65 6e 00 6d 65 73
00000150: 73 61 67 65 5f 65 6e 64 00 6d 65 73 73 61 67 65
00000160: 5f 73 74 61 72 74 00 63 75 73 74 6f 6d 5f 65 6e
00000170: 74 72 79 00
```

```
000000e0: 01 00 00 00 00 00 f1 ff 0e 00 00 00 00 00 00 00
000000f0: 00 00 00 00 00 00 00 00 0d 00 00 00 00 00 03 00
00000100: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110: 19 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00
```

Elf64_Sym.st_info (1/2) binding

- Поље st_info представља атрибуте симбола који описују његов:
 - начин везивања и
 - тип
- *ELF* формат за начин повезивања симбола дефинише следеће:

```
#define ELF64_ST_INFO(bind, type) (((bind) << 4) + ((type) & 0xf))
#define ELF64_ST_BIND(val)      (((unsigned char) (val)) >> 4)
#define STB_LOCAL      0      /* Local symbol */
#define STB_GLOBAL     1      /* Global symbol */
#define STB_WEAK       2      /* Weak symbol */
...
```

```
000000e0: 01 00 00 00 00 00 f1 ff 0e 00 00 00 00 00 00 00
000000f0: 00 00 00 00 00 00 00 00 0d 00 00 00 00 00 03 00
00000100: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110: 19 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00
```

Elf64_Sym.st_info (2/2) type

- Поље st_info представља атрибуте симбола који описују његов:
 - начин везивања и
 - тип
- *ELF* формат за тип симбола дефинише следеће:

```
#define ELF64_ST_INFO(bind, type) (((bind) << 4) + ((type) & 0xf))
#define ELF32_ST_TYPE(val)      ((val) & 0xf)
#define STT_NOTYPE 0 /* Symbol type is unspecified */
#define STT_OBJECT 1 /* Symbol is a data object */
#define STT_FUNC 2 /* Symbol is a code object */
#define STT_SECTION 3 /* Symbol associated with a section */
#define STT_COMMON 5 /* Symbol is a common data object */
...
```

```
000000e0: 01 00 00 00 00 00 f1 ff 0e 00 00 00 00 00 00 00
000000f0: 00 00 00 00 00 00 00 00 0d 00 00 00 00 00 03 00
00000100: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110: 19 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00
```

Elf64_Sym.st_other

- Поље st_other тренутно дефинише видљивост симбола
- *ELF* формат дефинише следеће могуће вредности:

```
#define STV_DEFAULT    0    /* Default symbol visibility rules */
#define STV_INTERNAL  1    /* Processor specific hidden class */
#define STV_HIDDEN     2    /* Sym unavailable in other modules */
#define STV_PROTECTED  3    /* Not preemptible, not exported */
```

```
000000e0: 01 00 00 00 00 00 f1 ff 0e 00 00 00 00 00 00 00
000000f0: 00 00 00 00 00 00 00 00 0d 00 00 00 00 00 00 00
00000100: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110: 19 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00
```

Elf64_Sym.st_shndx (1/2)

- Поље st_shndx представља индекс заглавља секције којој припада посматрани симбол

```
000000e0: 01 00 00 00 00 00 f1 ff 0e 00 00 00 00 00 00 00
000000f0: 00 00 00 00 00 00 00 00 0d 00 00 00 00 00 03 00
00000100: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110: 19 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00
```

Elf64_Sym.st_shndx (2/2)

- Посматрани симбол `message_end` припада секцији имена `.data` што је могуће установити на основу приказаног садржаја *section name string table* и **трећег улаза** у *section header table*

```
00000190: 00 2e 73 79 6d 74 61 62 00 2e 73 74 72 74 61 62
000001a0: 00 2e 73 68 73 74 72 74 61 62 00 2e 72 65 6c 61
000001b0: 2e 74 65 78 74 00 2e 64 61 74 61 00 2e 62 73 73
000001c0: 00
```

...

```
00000288: 26 00 00 00 01 00 00 00 03 00 00 00 00 00 00 00
00000298: 00 00 00 00 00 00 00 00 6e 00 00 00 00 00 00 00
000002a8: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000002b8: 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
000000e0: 01 00 00 00 00 00 00 f1 ff 0e 00 00 00 00 00 00 00
000000f0: 00 00 00 00 00 00 00 00 0d 00 00 00 00 00 03 00
00000100: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110: 19 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00
```

Elf64_Sym.st_value

- Поље `st_value` представља вредност посматраног симбола
- Интерпретација поља `st_value` зависи од типа *ELF* датотеке:
 - у *relocatable ELF* датотекама, поље `st_value` садржи померај од почетка секције до датог дефинисаног симбола
 - у *relocatable ELF* датотекама, поље `st_value` садржи поравнање симбола из секције са индексом заглавља `SHN_COMMON`
 - у *executable* и *shared object ELF* датотекама, поље `st_value` садржи виртуелну адресу

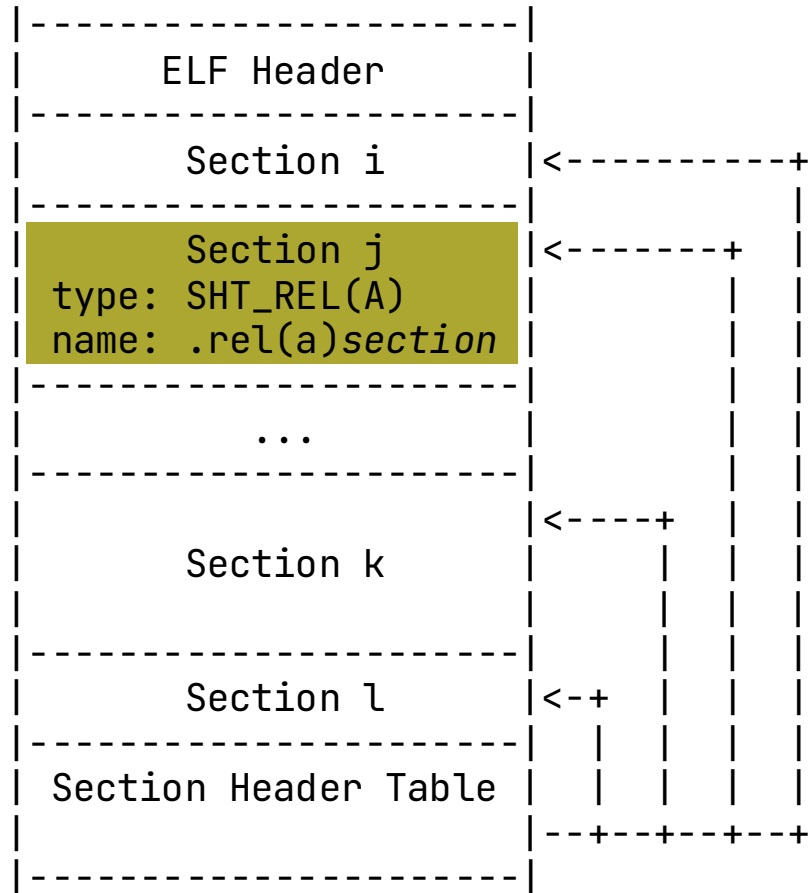
```
000000e0: 01 00 00 00 00 00 f1 ff 0e 00 00 00 00 00 00 00
000000f0: 00 00 00 00 00 00 00 00 0d 00 00 00 00 00 03 00
00000100: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110: 19 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00
```

Elf64_Sym.st_size

- Поље `st_size` дефинише величину додељену симболу; уколико симбол нема величину или величина симбола није позната ово поље има вредност 0 (нула)

```
000000e0: 01 00 00 00 00 00 f1 ff 0e 00 00 00 00 00 00 00
000000f0: 00 00 00 00 00 00 00 00 0d 00 00 00 00 00 03 00
00000100: 0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000110: 19 00 00 00 00 00 03 00 00 00 00 00 00 00 00 00
```

Relocation Table



Relocation Table

- *Relocation table* је **секција** типа `SHT_REL(A)` чији је садржај табела са улазима фиксних величина који су типа `Elf64_Rel(a)` (сваки појединачни улаз представља један релокациони запис)
- На основу садржаја заглавља секције типа `SHT_REL(A)` могуће је добити следеће податке:
 - `Elf64_Shdr.sh_offset` је померај од почетка *ELF* датотеке до локације где почиње *relocation table*
 - `Elf64_Shdr.sh_size` је величина секције *relocation table* изражено у броју бајтова
 - `Elf64_Shdr.sh_link` је индекс заглавља секције придружене табеле симбола
 - `Elf64_Shdr.sh_info` је индекс заглавља секције унутар које се врше преправке дефинисане посматраним релокацијама
 - `Elf64_Shdr.sh_entsize` је величина једног релокационог записа односно величина једног улаза унутар *relocation table*

Relocation Table

- Типови секција `SHT_RELA` и `SHT_REL` разликују се само по томе да ли релокациони записи унутар секција тог типа поседују експлицитно поље за непосредну вредност која се користи односно додаје приликом преправке (релокације):
 - секције типа `SHT_RELA` садрже улазе типа `Elf64_Rela` који имају поље за непосредну вредност коришћену приликом преправке
 - секције типа `SHT_REL` садрже улазе типа `Elf64_Rel` који немају поље за непосредну вредност коришћену приликом преправке (потребна вредност добија се из локације коју треба преправити)

Relocation Table

- На основу вредности поља `Elf64_Shdr.sh_offset` унутар заглавља неке секције типа `SHT_REL(A)` (*ELF* датотека може имати већи број секција типа `SHT_REL(A)`) добија се информација где се налази посматрана *relocation table* гледано у односу на почетак *ELF* датотеке
- Садржај једне *relocation table* у *relocatable ELF* датотеци *hello.o*, где унутар заглавља секције типа `SHT_RELA` поље `Elf64_Shdr.sh_offset` има вредност `0x178` а поље `Elf64_Shdr.sh_size` има вредност `0x18`, приказан је на следећем слајду

Relocation Table (hello.o)

00000178:	11	00	00	00	00	00	00	00	0b	00	00	00	02	00	00	00
00000188:	00	00	00	00	00	00	00	00								

Elf64_Rel(a)

```
typedef struct {  
    Elf64_Addr    r_offset;  
    Elf64_Xword   r_info;  
    Elf64_Sxword  r_addend;  
} Elf64_Rela;
```

```
typedef struct {  
    Elf64_Addr    r_offset;  
    Elf64_Xword   r_info;  
    /* addend from machine code */  
} Elf64_Rel;
```

Elf64_Rela

- Поља структуре `Elf64_Rela`, која чине један релокациони запис, описаћемо појединачно на примеру једног конкретног релокационог записа из *ELF* датотеке *hello.o*
- Садржај структуре `Elf64_Rela` у *relocatable ELF* датотеци *hello.o* која одговара улазу у *relocation table* са индексом 0 (нула) приказан је на дну слајда

```
00000178: 11 00 00 00 00 00 00 00 0b 00 00 00 02 00 00 00
00000188: 00 00 00 00 00 00 00 00
```

Elf64_Rela.r_offset

- Поље `r_offset` дефинише локацију где треба извршити преправку
- Интерпретација поља `r_offset` зависи од типа *ELF* датотеке:
 - у *relocatable ELF* датотекама, поље `r_offset` садржи померај од почетка секције до локације где треба извршити преправку
 - у *executable* и *shared object ELF* датотекама, поље `r_offset` садржи виртуелну адресу локације где треба извршити преправку

```
00000178: 11 00 00 00 00 00 00 00 0b 00 00 00 02 00 00 00
00000188: 00 00 00 00 00 00 00 00
```

Elf64_Rela.r_info

- Поље `r_info` дефинише
 - симбол према којем треба извршити релокацију (наводећи индекс тог симбола у табели симбола) и
 - тип релокације (специфично за посматрани процесор)
- *ELF* формат дефинише следеће:

```
#define ELF64_R_SYM(i)          ((i) >> 32)
#define ELF64_R_TYPE(i)        ((i) & 0xffffffff)
#define ELF64_R_INFO(sym,type) (((Elf64_Xword) (sym)) << 32) + (type))
#define R_X86_64_PC32    2    /* PC relative 32 bit signed */
#define R_X86_64_PLT32   4    /* 32-bit PLT address */
#define R_X86_64_32S     11   /* Direct 32-bit sign extended */
...
```

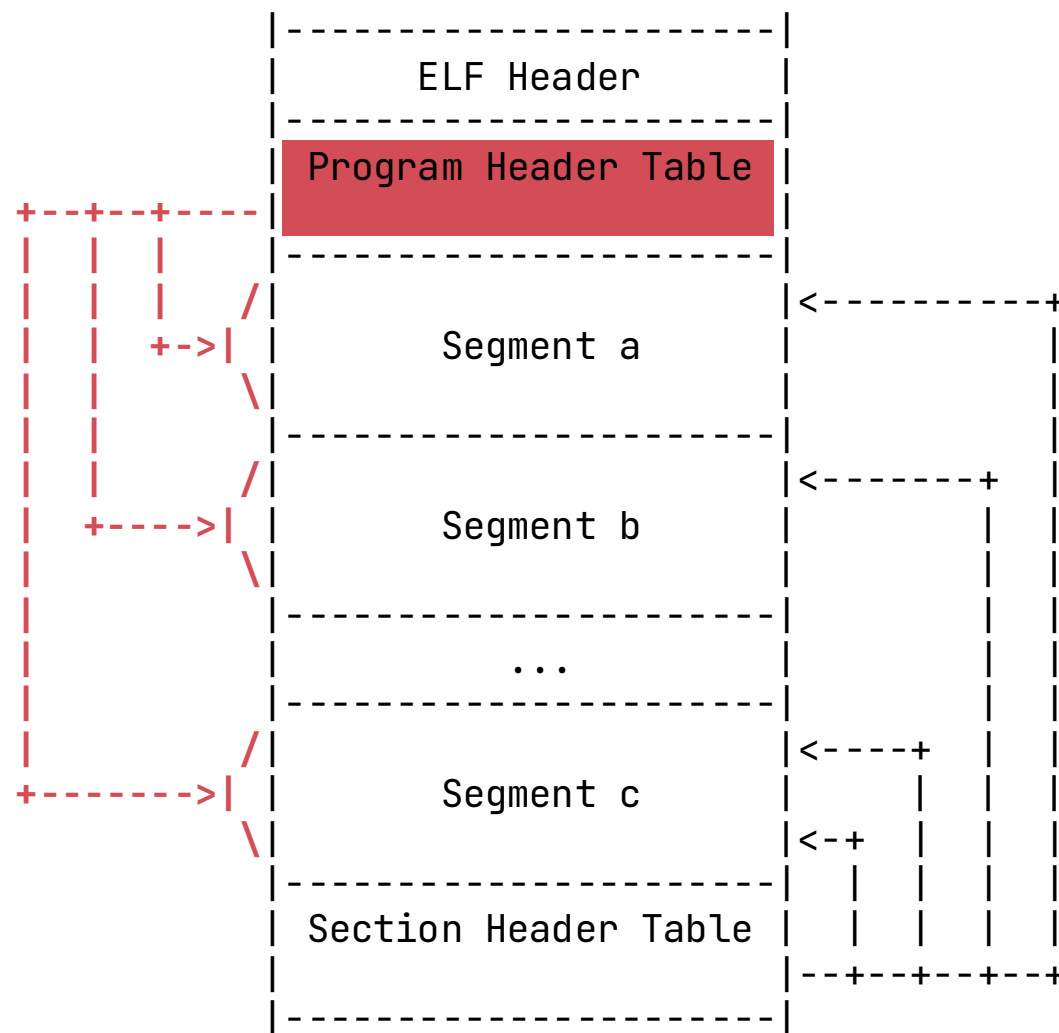
```
00000178: 11 00 00 00 00 00 00 00 0b 00 00 00 02 00 00 00
00000188: 00 00 00 00 00 00 00 00
```

Elf64_Rela.r_addend

- Поље r_addend дефинише непосредну вредност која се користи односно додаје приликом преправке (релокације)

```
00000178: 11 00 00 00 00 00 00 00 0b 00 00 00 02 00 00 00
00000188: 00 00 00 00 00 00 00 00
```

Program Header Table



Program Header Table

- ***Program header table*** представља низ елемената типа `Elf64_Phdr`
- Елементи *program header table* уобичајено се називају „улази“; сваки улаз представља заглавље једног дела програма који се састоји од једне или више секција и представља:
 - меморијски сегмент или
 - информације потребне систему за припрему програма за извршавање
- На основу садржаја *ELF header* могуће је добити следеће податке:
 - `Elf64_Ehdr.e_phoff` је померај од почетка *ELF* датотеке до локације где почиње *program header table*
 - `Elf64_Ehdr.e_phnum` је број елемената односно улаза унутар *program header table*
 - `Elf64_Ehdr.e_phentsize` је величина једног елемента односно улаза унутар *program header table*

Program Header Table

- Заглавља програма су значајна и имају смисла једино у оквиру *executable* и *shared object ELF* датотека тако да за наш пример *relocatable ELF* датотеке *hello.o* морамо прво извршити повезивање како бисмо добили, на пример, *executable ELF* датотеку *hello*
- *Executable ELF* датотеку *hello* могуће је добити на следећи начин:
`ld --entry=custom_entry -o hello hello.o`

Program Header Table

- На основу вредности поља `Elf64_Ehdr.e_phoff` добија се информација где се налази *program header table* гледано у односу на почетак *ELF* датотеке
- Садржај *program header table* у *executable ELF* датотеци *hello*, чије поље `Elf64_Ehdr.e_phoff` има вредност `0x40`, приказан је на следећим слајдовима

Program Header Table (hello)

00000040:	01	00	00	00	04	00	00	00
00000050:	00	00	40	00	00	00	00	00
00000060:	e8	00	00	00	00	00	00	00
00000070:	00	10	00	00	00	00	00	00
00000080:	00	10	00	00	00	00	00	00
00000090:	00	10	40	00	00	00	00	00
000000a0:	2e	00	00	00	00	00	00	00
000000b0:	01	00	00	00	06	00	00	00
000000c0:	00	20	40	00	00	00	00	00
000000d0:	0e	00	00	00	00	00	00	00
000000e0:	00	10	00	00	00	00	00	00

00	00	00	00	00	00	00	00
00	00	40	00	00	00	00	00
e8	00	00	00	00	00	00	00
01	00	00	00	05	00	00	00
00	10	40	00	00	00	00	00
2e	00	00	00	00	00	00	00
00	10	00	00	00	00	00	00
00	20	00	00	00	00	00	00
00	20	40	00	00	00	00	00
0e	00	00	00	00	00	00	00

Elf64_Phdr

```
typedef struct {  
    Elf64_Word    p_type;  
    Elf64_Word    p_flags;  
    Elf64_Off     p_offset;  
    Elf64_Addr    p_vaddr;  
    Elf64_Addr    p_paddr;  
    Elf64_Xword   p_filesz;  
    Elf64_Xword   p_memsz;  
    Elf64_Xword   p_align;  
} Elf64_Phdr;
```

Elf64_Phdr

- Поља структуре `Elf64_Phdr`, која су заглавље једног дела програма, описаћемо појединачно на примеру једног конкретног заглавља програма из *ELF* датотеке *hello*
- Садржај структуре `Elf64_Phdr` у *executable ELF* датотеци *hello* која одговара улазу у *program header table* са индексом 1 (један) приказан је на дну слајда

```
00000070: 00 10 00 00 00 00 00 00
00000080: 00 10 00 00 00 00 00 00
00000090: 00 10 40 00 00 00 00 00
000000a0: 2e 00 00 00 00 00 00 00
```

```
01 00 00 00 05 00 00 00
00 10 40 00 00 00 00 00
2e 00 00 00 00 00 00 00
00 10 00 00 00 00 00 00
```

Elf64_Phdr.p_type

- Поље `p_type` је тип **дела програма** којим се дефинише:
 - врста меморијског сегмента описаног датим заглављем или
 - начин интерпретације информација описаних датим заглављем
- *ELF* формат дефинише следеће могуће вредности:

```
#define PT_NULL      0  /* Program header table entry unused */
#define PT_LOAD      1  /* Loadable program segment */
#define PT_DYNAMIC    2  /* Dynamic linking information */
#define PT_INTERP     3  /* Program interpreter */
#define PT_NOTE       4  /* Auxiliary information */
#define PT_SHLIB      5  /* Reserved */
#define PT_PHDR       6  /* Entry for header table itself */
...
```

```
00000070:  00 10 00 00 00 00 00 00 01 00 00 00 05 00 00 00
00000080:  00 10 00 00 00 00 00 00 00 10 40 00 00 00 00 00
00000090:  00 10 40 00 00 00 00 00 2e 00 00 00 00 00 00 00
000000a0:  2e 00 00 00 00 00 00 00 00 10 00 00 00 00 00 00
```

Elf64_Phdr.p_flags

- Поље p_flags су атрибути релевантни за меморијски сегмент
- *ELF* формат дефинише следеће могуће вредности:

```
#define PF_X          (1 << 0)  /* Segment is executable */
#define PF_W          (1 << 1)  /* Segment is writable */
#define PF_R          (1 << 2)  /* Segment is readable */
#define PF_MASKOS     0x0ff00000 /* OS-specific */
#define PF_MASKPROC   0xf0000000 /* Processor-specific */
```

```
00000070: 00 10 00 00 00 00 00 00 00 01 00 00 00 05 00 00 00
00000080: 00 10 00 00 00 00 00 00 00 00 10 40 00 00 00 00 00
00000090: 00 10 40 00 00 00 00 00 00 2e 00 00 00 00 00 00 00
000000a0: 2e 00 00 00 00 00 00 00 00 00 10 00 00 00 00 00 00
```

Elf64_Phdr.p_offset

- Поље `p_offset` представља померај од почетка *ELF* датотеке до првог бајта посматраног дела програма изражено у броју бајтова

```
00000070: 00 10 00 00 00 00 00 00 01 00 00 00 05 00 00 00
00000080: 00 10 00 00 00 00 00 00 00 10 40 00 00 00 00 00
00000090: 00 10 40 00 00 00 00 00 2e 00 00 00 00 00 00 00
000000a0: 2e 00 00 00 00 00 00 00 00 10 00 00 00 00 00 00
```

Elf64_Phdr.p_vaddr

- Поље `p_vaddr` је виртуелна адреса на којој ће се у меморији наћи први бајт меморијског сегмента описаног датим заглављем (предвиђена виртуелна адреса)

00000070:	00	10	00	00	00	00	00	00	01	00	00	00	05	00	00	00
00000080:	00	10	00	00	00	00	00	00	00	10	40	00	00	00	00	00
00000090:	00	10	40	00	00	00	00	00	2e	00	00	00	00	00	00	00
000000a0:	2e	00	00	00	00	00	00	00	00	10	00	00	00	00	00	00

Elf64_Phdr.p_paddr

- Поље `p_paddr` је физичка адреса на којој ће се у меморији наћи први бајт меморијског сегмента описаног датим заглављем (предвиђена физичка адреса за системе у којима то има смисла)

00000070:	00 10 00 00 00 00 00 00	01 00 00 00 05 00 00 00
00000080:	00 10 00 00 00 00 00 00	00 10 40 00 00 00 00 00
00000090:	00 10 40 00 00 00 00 00	2e 00 00 00 00 00 00 00
000000a0:	2e 00 00 00 00 00 00 00	00 10 00 00 00 00 00 00

Elf64_Phdr.p_filesz

- Поље `p_filesz` представља величину простора изражено у бајтовима који дати **део програма** заузима у *ELF* датотеци (може бити једнак нули)

```
00000070: 00 10 00 00 00 00 00 00 01 00 00 00 05 00 00 00
00000080: 00 10 00 00 00 00 00 00 00 10 40 00 00 00 00 00
00000090: 00 10 40 00 00 00 00 00 2e 00 00 00 00 00 00 00
000000a0: 2e 00 00 00 00 00 00 00 00 10 00 00 00 00 00 00
```

Elf64_Phdr.p_memsz

- Поље `p_memsz` представља величину простора изражено у бајтовима који дати **део програма** заузима у адресном простору (може бити једнак нули)

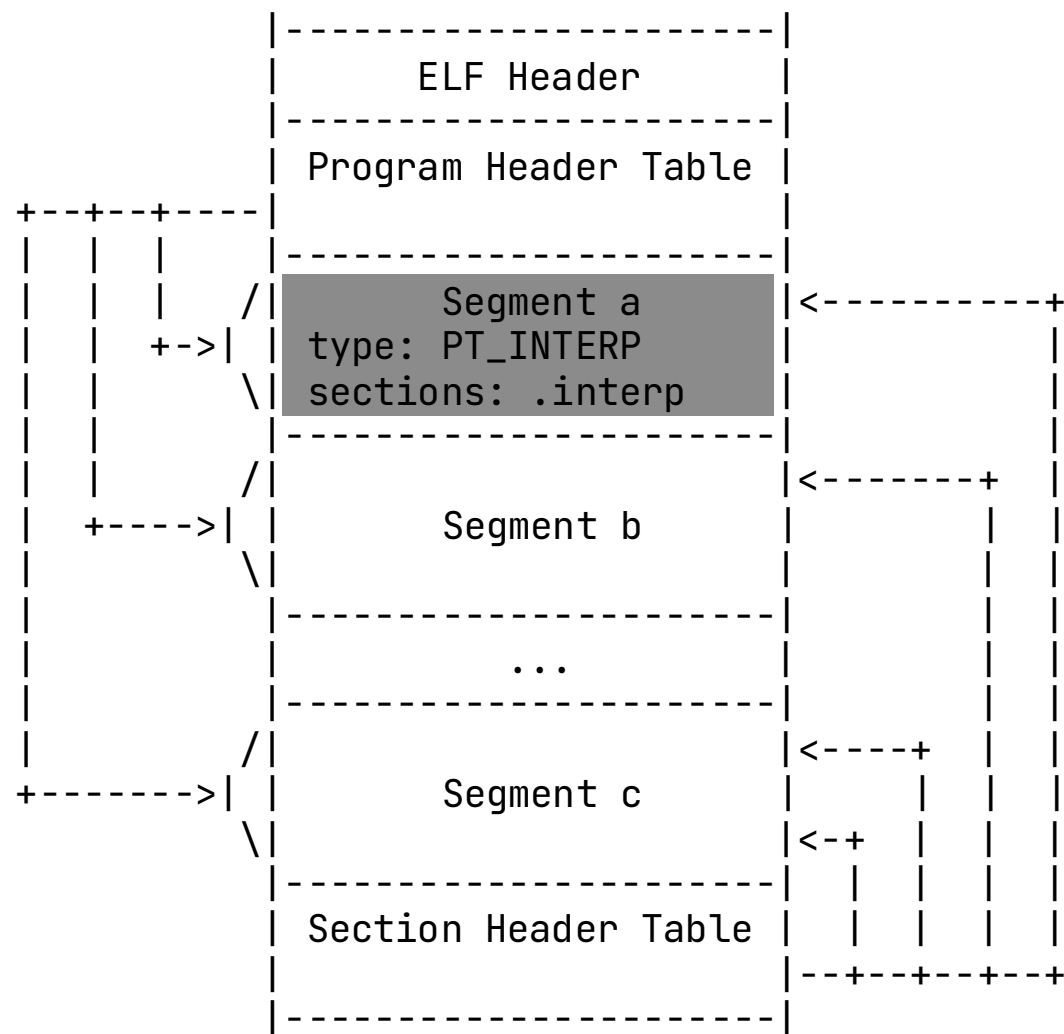
```
00000070: 00 10 00 00 00 00 00 00 01 00 00 00 05 00 00 00
00000080: 00 10 00 00 00 00 00 00 00 10 40 00 00 00 00 00
00000090: 00 10 40 00 00 00 00 00 2e 00 00 00 00 00 00 00
000000a0: 2e 00 00 00 00 00 00 00 00 10 00 00 00 00 00 00
```

Elf64_Phdr.p_align

- Поље `p_align` дефинише поравнање датог **дела програма**
 - нула или један значи да није захтевано поравнање
 - поред нула и један дозвољене су само вредности које су степен двојке

```
00000070: 00 10 00 00 00 00 00 00 01 00 00 00 05 00 00 00
00000080: 00 10 00 00 00 00 00 00 00 10 40 00 00 00 00 00
00000090: 00 10 40 00 00 00 00 00 2e 00 00 00 00 00 00 00
000000a0: 2e 00 00 00 00 00 00 00 00 10 00 00 00 00 00 00
```

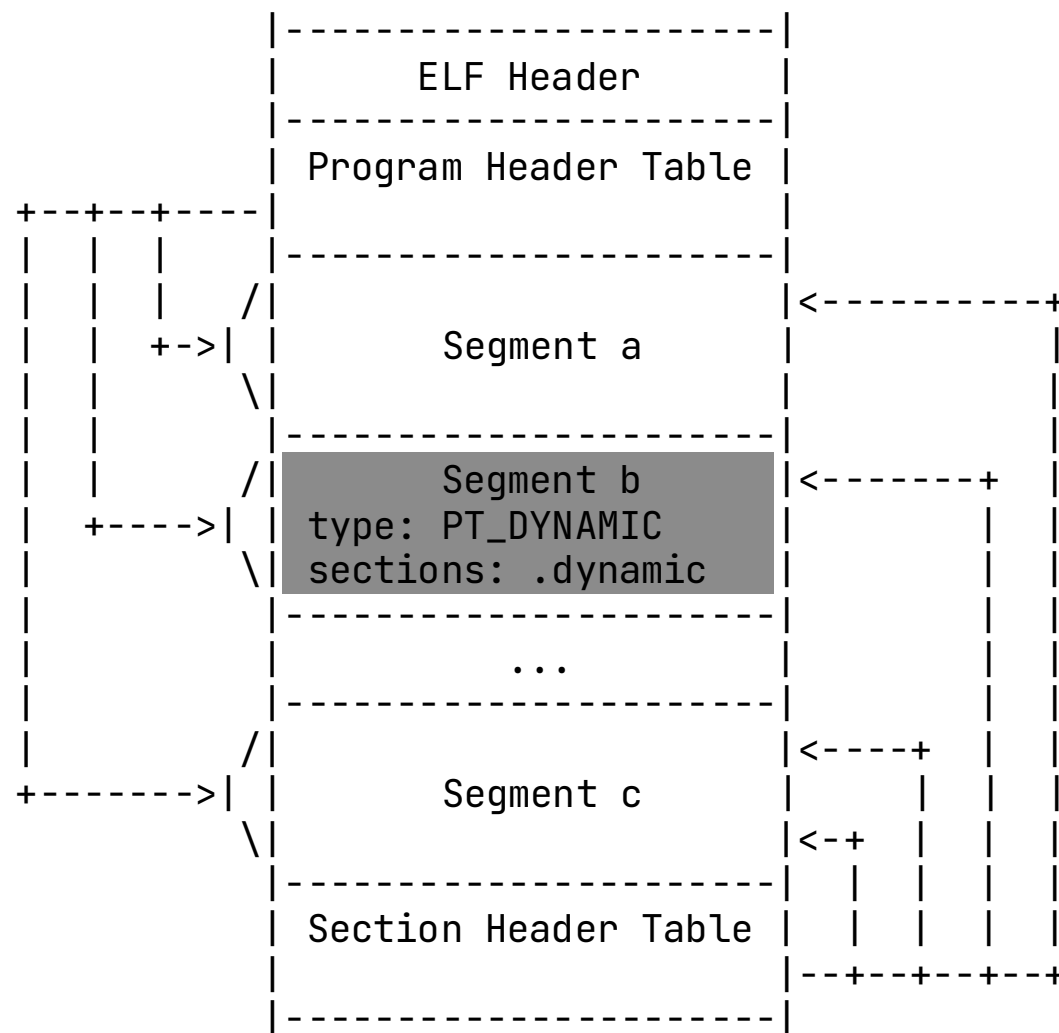
Program Interpreter



Program Interpreter

- *Executable ELF* датотека која учествује у динамичком повезивању мора имати једно **заглавље дела програма** типа `PT_INTERP`
- Приликом системског позива `exec()` за покретање програма систем из **дела програма** типа `PT_INTERP` узима назив интерпретера и уместо програма учитава интерпретер у адресни простор па је задатак интерпретера да изврши:
 - учитавање меморијских сегмената програма,
 - учитавање библиотека са динамичким везивањем у случају потребе уз њихово мапирање у адресни простор посматраног процеса
 - повезивање програма са динамичким библиотекама
- Динамички линкер *ld-linux.so* најчешће обавља задатак интерпретера

Dynamic Section



Dynamic Section

- *Dynamic section* је **секција** типа `SHT_DYNAMIC`, обухваћена **делом програма** типа `PT_DYNAMIC`, чији је садржај низ података типа `Elf64_Dyn` (елементи низа су подаци потребни динамичком линкеру)
- Садржај *dynamic section* генерише статички линкер како би динамичком линкеру пружио неопходне податке
- *ELF* датотека која учествује у динамичком повезивању мора имати **заглавље дела програма** типа `PT_DYNAMIC`

Elf64_Dyn

```
typedef struct {  
    Elf64_Sxword d_tag;  
    union {  
        Elf64_Xword d_val;  
        Elf64_Addr  d_ptr;  
    } d_un;  
} Elf64_Dyn;  
  
extern Elf64_Dyn _DYNAMIC[];
```

Elf64_Dyn.d_tag (1/3)

- Поље d_tag говори како треба интерпретирати поље d_un посматраног елемента низа из *dynamic section*
- Неке од вредности и тумачења за поље d_tag, дефинисано *ELF* форматом, дати су на следећим слајдовима

Elf64_Dyn.d_tag (2/3)

Назив	В.	d_un	exe	.so	Опис
DT_NULL	0	ign	man	man	Овај елемент означава крај низа _DYNAMIC
DT_NEEDED	1	d_val	opt	opt	Померај у табели стрингова до имена коришћене библиотеке
DT_STRTAB	5	d_ptr	man	man	Адреса стринг табеле у којој су имена симбола, имена библиотека итд.
DT_SYMTAB	6	d_ptr	man	man	Адреса табеле симбола
DT_REL DT_RELA	17 7	d_ptr	man	opt	Адреса табеле релокација
DT_RELSZ DT_RELASZ	18 8	d_val	man	opt	Величина читаве DT_REL(A) табеле релокације
DT_RELENT DT_RELAENT	19 9	d_val	man	opt	Величина једног улаза DT_REL(A) табеле релокације

Elf64_Dyn.d_tag (3/3)

Назив	В.	d_un	exe	.so	Опис
DT_STRSZ	10	d_val	man	man	Величина читаве стринг табеле
DT_SYMENT	11	d_val	man	man	Величина једног улаза у стринг табелу
DT_SONAME	14	d_val	ign	opt	Померај у DT_STRTAB табелу стрингова до имена посматраног <i>shared object</i>
DT_INIT_ARRAY	25	d_ptr	opt	opt	Адреса низа показивача на иницијализационе функције
DT_FINI_ARRAY	26	d_ptr	opt	opt	Адреса низа показивача на завршне функције
DT_RUNPATH	29	d_val	opt	opt	Померај у DT_STRTAB табелу стрингова до путање за претрагу библиотека
DT_FLAGS	30	d_val	opt	opt	Флегови специфични за објектни програм који се учитава

Elf64_Dyn.d_un

- Поље d_un представља унију података чија интерпретација зависи од вредности поља d_tag:
 - поље d_val садржи одговарајућу целобројну вредност
 - поље d_ptr садржи одговарајућу виртуелну адресу